A Primer on Local-First Document Intelligence

 It exists solely to give the Phase 0 spike a bundled, text-based, public-domain PDF to parse, chunk, embed, store, and retrieve against, and contains deliberately specific factual statements so an engineer can ask a grounded question and verify that the answer comes from the retrieved passages rather than from a language model's own knowledge.

Section 1: What "Local-First" Means

Local-first software is software that treats the user's own device as the primary place where computation and storage happen, rather than treating a remote server as the source of truth. In a local-first document assistant, the text of a document, the embeddings computed from that text, and the index used to search those embeddings all live on the user's machine. A network connection may still be useful — for downloading a model once, or for an optional cloud fallback — but it is not required for the core workflow of asking a question about a document you already have open.

The appeal of this approach is threefold. First, it is private by construction: if a document never leaves the device, there is no server-side log, no third-party processor, and no data breach surface for that document's contents. Second, it is resilient: a local-first tool keeps working on an airplane, in a basement server room with no signal, or during a provider outage. Third, it is often faster for small to medium workloads, because there is no round trip to a remote data center for every keystroke or every question.

Local-first is not the same as "offline-only." A well-designed local-first system can still synchronize, still call out to a remote model when the user explicitly asks for a more capable one, and still check for software updates. The defining property is that the default path — the one exercised most often — does not require a network round trip, and the user can always tell which path they are on.

Section 2: The Four Pillars of an In-Browser Retrieval Pipeline

An in-browser document question-answering pipeline can be decomposed into four pillars: parsing, chunking, embedding, and storage. Each pillar is a distinct engineering problem with its own failure modes, and a robust pipeline treats each one as swappable.

Parsing turns a raw file — most often a PDF — into plain text. This sounds simple but is not: PDFs encode text as positioned glyphs on a page rather than as a stream of characters, so a parser must reconstruct reading order, handle multi-column layouts, and decide what to do with tables, images, and scanned (image-only) pages. A parser that cannot find a text layer should say so plainly rather than silently returning an empty string.

Chunking splits the extracted text into pieces small enough for an embedding model to process, and large enough to preserve useful context. The chunk size is usually expressed in tokens, not characters, because the embedding model's context window is defined in tokens. A chunker that ignores the tokenizer and estimates size by character count alone will sometimes produce chunks that silently get truncated by the embedding model — a subtle failure that this Phase 0 spike is specifically designed to catch and record.

Embedding converts each chunk of text into a fixed-length vector of floating point numbers, chosen so that chunks with similar meaning end up close together in that vector space. A single sentence and a full paragraph making the same point should embed to nearby vectors even though their word counts differ substantially. The quality of this vector space determines the quality of everything downstream: retrieval, ranking, and ultimately the grounding of a generated answer.

Storage keeps the chunks, their vectors, and enough metadata to trace a retrieved chunk back to its source location in the original document. In a browser, this storage must survive a page reload without forcing the user to wait through re-parsing and re-embedding a second time, because that wait is the single biggest threat to the feeling that the tool is "yours" rather than a service you are renting by the session.

Section 3: Why Vector Search Alone Is Not Enough

Vector similarity search finds chunks whose embeddings are close to the embedding of a question. This works remarkably well for paraphrase and conceptual matches — a question phrased very differently from the source text can still retrieve the right passage, because the embedding model captures meaning rather than exact wording. But vector search has known weak spots.

It struggles with rare, exact tokens: model numbers, part codes, unusual proper nouns, and numeric identifiers often do not embed distinctively, because the embedding model was trained to generalize over meaning rather than to memorize exact strings. A keyword or full-text index (commonly built on an inverted index or a database's built-in full-text search) is much better at exact-token recall. This is why a mature retrieval system frequently combines a vector index with a lexical index — sometimes called hybrid search — and merges the two ranked lists before handing the top results to a generation step. Hybrid search is explicitly out of scope for this Phase 0 walking skeleton; it is deferred to a later slice once the vector-only path has been proven end to end.

Vector search also has no inherent concept of recency, authority, or document structure. A retrieval system built only on cosine similarity treats a footnote and a section heading as equally weighted text, unless additional signals (like the store's document and chunk metadata) are layered on top during ranking.

Section 4: Index Strategies for Approximate Nearest Neighbor Search

At small scale — a handful of documents and a few thousand chunks — an exact nearest-neighbor scan, which compares the query vector against every stored vector, is entirely fast enough. At larger scale, an exact scan becomes the bottleneck, and systems switch to an approximate index structure that trades a small amount of recall for a large speedup.

HNSW, short for Hierarchical Navigable Small World graphs, is one of the most widely used approximate index structures for vector search. It builds a multi-layer

graph where each layer is a sparser version of the layer below, so that a query can quickly navigate from a coarse starting point down to a small neighborhood of genuinely close vectors without ever comparing against the whole dataset. HNSW is popular because it offers a strong recall-versus-speed tradeoff and does not require the index to be rebuilt from scratch every time new vectors are added, unlike some older approximate methods.

Whether a given database engine can build an HNSW index depends on the extension providing vector support and the exact build the engine ships. This Phase 0 spike treats the answer to that question as an empirical result to record, not an assumption to make in advance: if HNSW index creation succeeds, the spike records that outcome; if it fails, the spike falls back to an exact scan and records that outcome instead. At the scale of a single personal document — dozens to a few hundred chunks — an exact scan is already fast enough that the choice between the two index strategies has no user-visible effect on this slice's retrieval latency target.

Section 5: The Cold-Start Problem for In-Browser Models

Running a machine learning model entirely inside a browser tab means the model's weights must reach the user's device before any inference can happen. A modestly sized embedding model can still be well over one hundred megabytes even after aggressive quantization, and a capable generation model is typically several times larger than that. On a slow connection, or the very first time a user opens the tool, this download can take anywhere from twenty seconds to several minutes.

This delay, sometimes called the cold-start problem, is widely considered the single biggest adoption risk for browser-native AI tools, because it sits directly in the critical path of a user's very first impression. Three mitigations are commonly used together. The first is caching: once downloaded, model weights should be written to persistent browser storage so that every subsequent visit — a warm start — skips the download entirely and only pays the cost of re-initializing the model runtime, which should take a small number of seconds rather than tens of seconds. The second is offering a smaller, faster model as a default or a fallback while a larger, more capable

model downloads in the background. The third is being honest with the user about what is happening: a progress indicator that names the file being downloaded and its size turns an unexplained multi-second freeze into a comprehensible, and therefore tolerable, wait.

A well-instrumented pipeline measures both the cold-start time and the warm-start time separately, because they answer different product questions. Cold-start time tells you how painful a brand-new user's first session will be. Warm-start time tells you how snappy the tool feels for a returning user, which is the experience that determines whether the tool earns a permanent place in someone's daily workflow.

Section 6: Making Privacy Observable Rather Than Merely True

A tool can be private in the sense that no document content is ever transmitted off the device, and still fail to feel private to its user, if that fact is never made visible. Observable privacy means the interface actively shows what is and is not happening on the network, rather than relying on a policy document the user is unlikely to read.

A practical way to make privacy observable is to route every network request an application makes through a single, central client, and to have that client record, for each request, its destination domain, its purpose, whether user content was included in the request body, and its outcome. That log can then be rendered directly in the interface in plain language: "Parsed locally," "Generated embeddings on this device," "No document content has left this device," "Downloaded model weights from a model registry." None of these statements require the user to open a browser's developer tools to verify; the application is simply telling the truth about itself, continuously, as it works.

This pattern has a second benefit beyond user trust: it also gives engineers and reviewers a single choke point to audit. If every content-bearing network call must pass through one function, then verifying that a given code path cannot leak document content becomes a matter of checking that the code path never calls that function with user content — a structural guarantee rather than a matter of reviewer

vigilance.

Section 7: Development-Only Cloud Fallbacks

Even a project firmly committed to local-first, on-device inference often benefits from a development-only cloud escape hatch. During early development, a full local generation model may not yet be integrated, may be too slow on the developer's hardware, or may simply not be the question being tested that day. A configurable connection to a general-purpose, OpenAI-compatible inference endpoint lets a developer keep iterating on the surrounding pipeline — parsing, chunking, storage, retrieval, and the interface — without being blocked on the local model work finishing first.

The engineering discipline this requires is keeping that escape hatch clearly separated from the production, on-device path, and making sure it is never silently used in place of the real thing. Three properties are worth enforcing together. The escape hatch should default to off, so a developer or reviewer running the tool for the first time sees the local behavior by default. Any use of the escape hatch should be visibly disclosed at the moment it happens, naming the destination the request is going to. And, over time, the code implementing the escape hatch should be excluded from production builds entirely, rather than merely disabled by a runtime flag that someone could accidentally leave on.

Section 8: Reading the Results of a Proof-of-Concept Honestly

The purpose of an early technical proof-of-concept is to convert assumptions into either confirmed facts or clearly documented open risks — not to produce a polished product. A proof-of-concept that only ever reports success is not doing its job; the valuable output of this kind of exercise is a small number of measured numbers and a small number of definitive yes-or-no answers to the riskiest technical questions a project is resting on.

For a browser-based retrieval pipeline, the riskiest questions typically concern persistence (does the local database survive a reload without losing data), search

behavior (does the chosen index type build successfully, and if not, does the fallback still perform acceptably), and model loading (how long does a first-time user wait, and how much shorter is that wait on a return visit). A proof-of-concept that answers these three questions with real, measured numbers — even if some of those numbers miss an aspirational target — has done more useful work than a demo that looks impressive but leaves every one of those questions unmeasured.

Section 9: A Short Glossary

Chunk: a contiguous span of text extracted from a document, sized to fit within an embedding model's input limit, small enough to be specific and large enough to retain context.

Embedding: a fixed-length numeric vector produced by a machine learning model from a piece of text, positioned in a high-dimensional space such that texts with similar meaning are close together.

Retrieval: the act of finding the stored chunks whose embeddings are closest to the embedding of a user's question, typically ranked by cosine similarity or a related distance measure.

Grounding: constraining a generated answer to be consistent with a specific set of retrieved source passages, rather than allowing a model to answer purely from its own trained-in knowledge.

Cold start: the first time a resource, such as a downloaded model, is initialized on a device, including the time spent fetching it from the network.

Warm start: a subsequent initialization of the same resource, using data already cached on the device, without a network fetch.

Section 10: A Note on the Number Five

For the purposes of testing retrieval in this Phase 0 spike, this primer records one

arbitrary, specific, and easily checkable fact: this document's default retrieval configuration for the Phase 0 spike used a value of five for the number of chunks retrieved per question. That default of five was chosen because it is large enough to give a generation step several independent passages to draw on, and small enough that a human reviewer can still read every retrieved passage in a few seconds when checking whether an answer is properly grounded. If asked what default number of chunks the Phase 0 spike retrieves per question, the correct, retrievable answer, present verbatim in this section, is five.

Section 11: Memory Pressure and Large Documents

A browser tab does not have unlimited memory, and a sufficiently large document can exhaust it well before a desktop application would notice any strain. The failure mode to avoid is a silent tab crash: a user who uploads an unusually large file and simply watches the page disappear has no way to know whether the file was too big, whether the model failed to load, or whether something else went wrong entirely.

A well-behaved pipeline sets a soft ceiling on document size, watches for signs of memory pressure during parsing and embedding, and reports a clear, specific message when that ceiling is approached or exceeded, rather than allowing the browser's own out-of-memory handling to be the first and only signal the user receives. This is a defensive engineering practice: it does not make the pipeline handle arbitrarily large documents, but it does make the boundary of what the pipeline can handle visible and explainable rather than a silent, unexplained failure.

Section 12: Browser Coverage and the Fallback Path

Not every browser, and not every machine within a browser that does support the relevant standard, provides equally solid GPU-accelerated inference in the browser. Coverage varies by browser vendor, by operating system, and even by graphics driver version on a given operating system, with some combinations — certain graphics driver and operating system pairings in particular — known to have rough edges.

Any tool that depends on GPU acceleration for its core workflow should treat a

slower, portable fallback path as a first-class citizen rather than an afterthought, because the alternative is a tool that quietly does not work at all for some meaningful fraction of visitors, with no indication of why. Detecting the unsupported case up front, and stating it plainly rather than failing partway through a long-running operation, turns a confusing dead end into an understandable, if less capable, experience.

Section 13: Editing and Retrieval as Complementary Workflows

Question answering over a document and editing a document are often built as two separate products, but they address the same underlying need: helping someone work with text they already have, rather than text a model invents from nothing. A reader asking "what does this paragraph mean" and a writer asking "can you rewrite this paragraph using the tone of the introduction" are both drawing on the same retrieval and grounding machinery — the difference is only in what is done with the retrieved context once it has been found.

Treating the writing surface as a first-class part of the product, rather than a bolt-on chat window next to a document viewer, keeps both workflows honest about where their answers come from: an inserted, cited sentence in a document and a chat-style answer to a question should be able to point back to the same underlying retrieved passages.

Section 14: Closing Summary

This primer has walked through the four pillars of an in-browser retrieval pipeline — parsing, chunking, embedding, and storage — and the surrounding concerns that make such a pipeline trustworthy and usable: honest handling of approximate search, honest handling of the cold-start problem, observable privacy, a clearly bounded development-only cloud fallback, and honest reporting of a proof-of-concept's actual, measured results rather than only its successes. None of these concerns are exotic; together they describe what it takes to make a browser-based document assistant feel like a private, dependable tool rather than a clever demo.

Section 15: Frequently Asked Questions

Does a local-first pipeline ever need a network connection at all? Yes, at least once: the models used for parsing, embedding, and generation typically need to be downloaded the first time they are used, unless they were bundled directly with the application, which is unusual given their size. After that first download, a well-cached pipeline can run its core workflow — opening a previously indexed document and asking it a question — without any further network access.

Why not just send the document to a server and let a much larger model handle everything? A server-side model can indeed be more capable, but it requires the document to leave the user's device, requires the provider to be reachable and to have not changed its terms of service, and introduces an ongoing operating cost that scales with usage. A local-first design trades some capability for privacy, reliability, and a cost structure that does not grow with how much a given user relies on the tool.

Is an approximate index like HNSW ever worse than doing nothing at all? An approximate index adds build time and memory overhead. At very small scale, that overhead can outweigh the benefit, because an exact scan over a few hundred vectors is already fast. This is exactly why a pipeline that is uncertain of its eventual scale should measure both paths rather than assuming the more sophisticated one is automatically the better choice.

What happens if a user asks a question with no good answer in the document? A trustworthy pipeline should say so, rather than manufacturing a plausible-sounding but ungrounded answer. Reporting "no strongly relevant passages were found" is a better outcome than silently returning the most similar passage in the store even when that passage is not actually relevant, because the former preserves the user's trust in every other answer the tool gives them.

Appendix A: Suggested Test Questions For This Document

The following questions are answerable directly from the sections above, and are suggested here for anyone manually verifying that the Phase 0 spike's retrieval and

generation steps are properly grounded in retrieved passages rather than in a language model's own background knowledge.

What does the abbreviation HNSW stand for? The answer, given in Section 4, is Hierarchical Navigable Small World graphs.

What default number of chunks does the Phase 0 spike retrieve per question? The answer, given in Section 10, is five.

According to this document, what are the four pillars of an in-browser retrieval pipeline? The answer, given in Section 2, is parsing, chunking, embedding, and storage.

What term does this document use for constraining a generated answer to be consistent with retrieved source passages? The answer, given in the glossary in Section 9, is grounding.

Appendix B: Revision Notes

This document was authored specifically for the AAFNO Phase 0 proof-of-concept, described in the project's specification and implementation plan (see specs/2026-07-05-phase0-poc-slice1.md and plans/2026-07-05-phase0-poc-slice1.md in the project repository). It is intentionally structured into short, self-contained sections and paragraphs so that a recursive text chunker can split it along natural boundaries — paragraph breaks, then sentence breaks — without cutting a sentence in half. It contains no images, no tables, and no scanned pages, so a parser should be able to extract its full text content directly from the PDF's text layer with no OCR step required.